or OpenAI, often through LangChain or the Vercel AI SDK. While these libraries can orchestrate tools and models, each system defines integrations in its own way, making reuse difficult.

The MCP simplifies this process by introducing a standardized way for AI applications to discover and invoke external tools and resources. Rather than wiring each tool manually, developers can expose them through MCP servers, while AI applications (or other clients) communicate using a shared protocol.

As AI adoption accelerates, one-off solutions quickly become a problem, since each application reinvents how tools are exposed to AI models. LangChain.js introduced tools and agents, but integrations built there don't easily transfer to other frameworks like LlamaIndex or the Vercel AI SDK. A developer might build a weather API integration for LangChain, but reusing it elsewhere requires starting from scratch because each framework has its own conventions, and assistants face inconsistent approaches to security, permissions, and resource management.

What's been missing is a common protocol—a shared language and structure for how AI systems discover, use, and compose tools. The MCP provides this missing standardization, enabling tools to become plug-and-play across assistants, frameworks, and applications.

Here we will apply the MCP specifically to chat-based web applications like Astra/React interfaces backed by Next.js, though the MCP itself is much broader and supports multiagent and distributed systems.

12.2 MCP architecture

At its core, the MCP establishes a client-server architecture that works as follows:

1. The MCP client (e.g., the AI agent or SDK) requests capabilities and invokes tools.
2. The MCP server exposes those tools, handles execution, and returns results.
3. Both communicate via a well-defined protocol, ensuring interoperability.

Figure 12. 1 shows how this is set up.

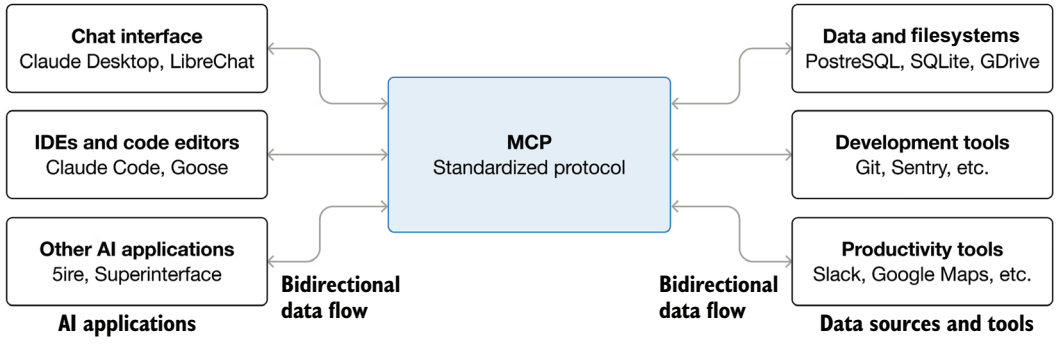


Figure 12.1 The MCP defines a standardized client–server interface that enables communication between web applications powered by LLMs (such as chat interfaces and code editors) and various external data sources and services (like databases, development tools, and productivity software). (Source: https://modelcontextprotocol.io/)